Divide-and-Conquer

https://kasets.art/kxXMOS

MODIFY FROM A. LEVITIN "INTRODUCTION TO THE DESIGN & ANALYSIS OF ALGORITHMS," 3RD ED., CH. 5 ©2012 PEARSON EDUCATION, INC. UPPER SADDLE RIVER, NJ.

Outline

- Divide-and-Conquer concept
- Binary tree traversals
- Multiplication of large integers
- Matrix multiplication: Strassen's algorithm
- Closest-pair and convex-hull algorithms

Divide-and-Conquer

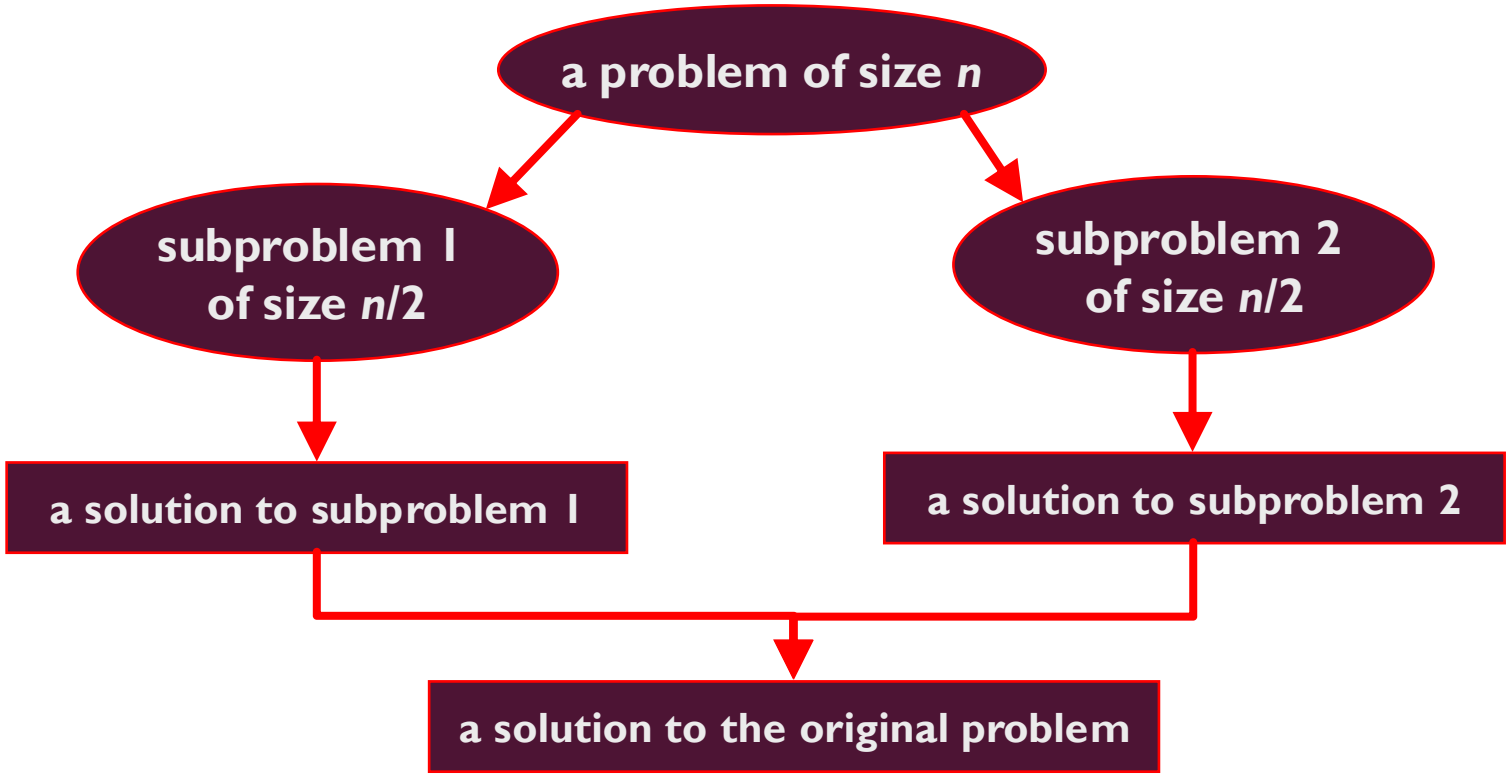
The most-well known algorithm design strategy:

1. Divide instance of problem into two or more smaller instances

2. Solve smaller instances recursively

3. Obtain solution to original (larger) instance by combining these solutions

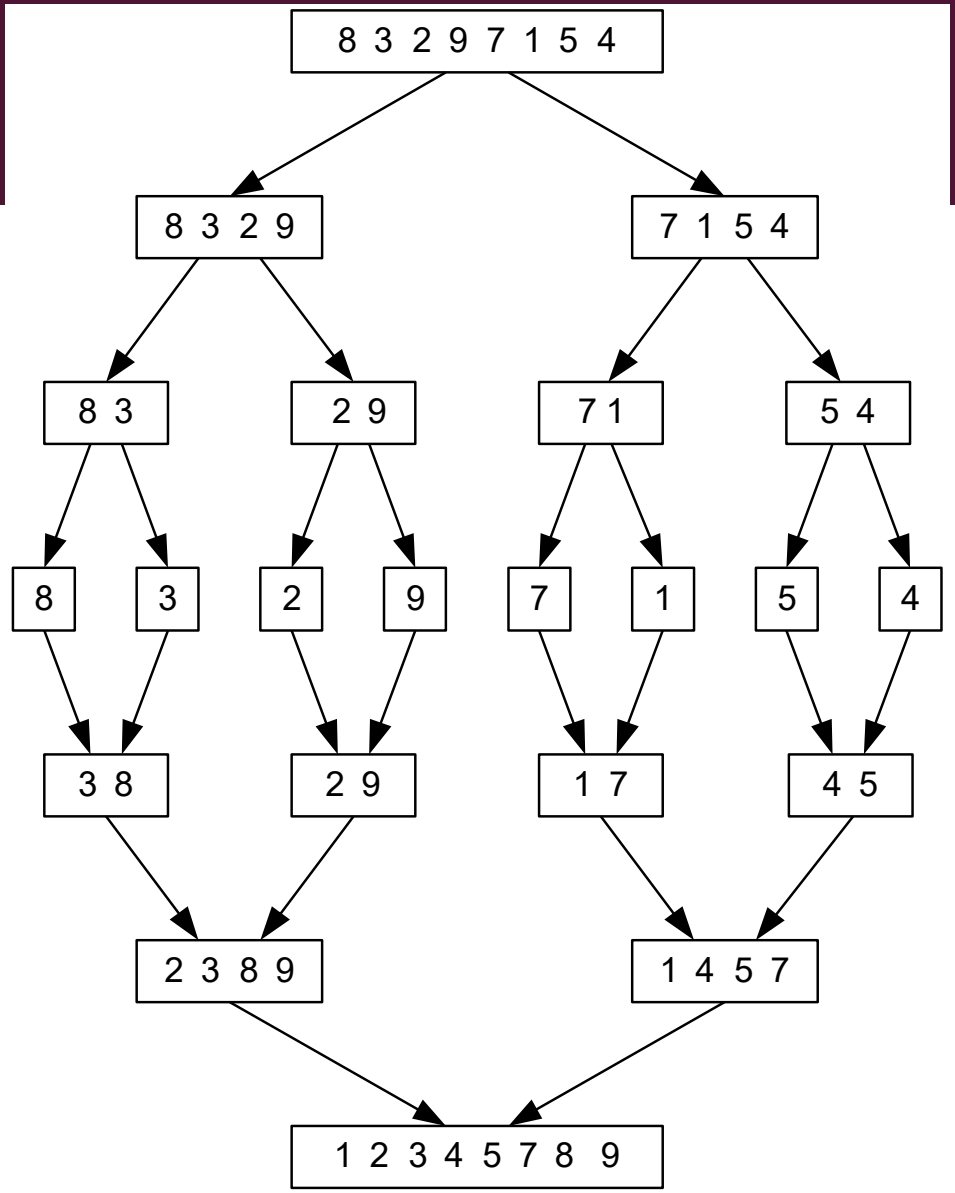
Divide-and-Conquer Technique (cont.)



Divide-and-Conquer Examples

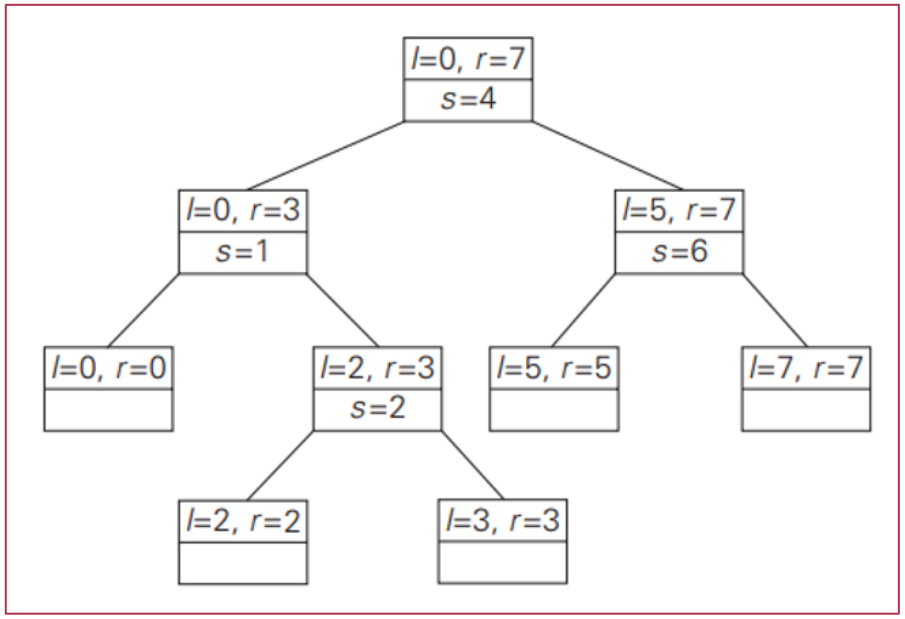
- Sorting: merge sort and quick sort
- Binary tree traversals
- Multiplication of large integers
- Matrix multiplication: Strassen's algorithm
- Closest-pair and convex-hull algorithms
- Binary search: decrease-by-half (or degenerate divide&conq.)

Merge sort Example



Quicksort Example

0	1	2	3	4	5	6	7
	i						*j*
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	2	8	9	7
				j	*i*		
5	3	1	4	2	8	9	7
2	3	1	4	**5**	8	9	7
	i		*j*				
2	3	1	4				
	i	*j*					
2	3	1	4				
	i	*j*					
2	1	3	4				
	j	*i*					
2	1	3	4				
1	**2**	3	4				
1							
			i j				
		3	4				
		j	*i*				
		3	4				
			4				
						i	*j*
					8	9	7
						i	*j*
					8	7	9
						j	*i*
					8	7	9
					7	**8**	9
					7		
							9



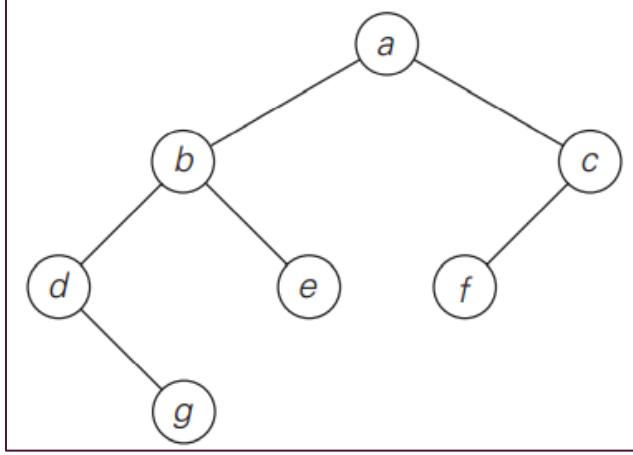
l:left, r: right, s: split position

Binary Tree Algorithms

- Binary tree is a divide-and-conquer ready structure!

- Ex. 1: Classic traversals (preorder, inorder, postorder)

```
Algorithm Inorder(T)
  if (T ≠ ∅)
     Inorder(T_left)
     print(root of T)
     Inorder(T_right)
```

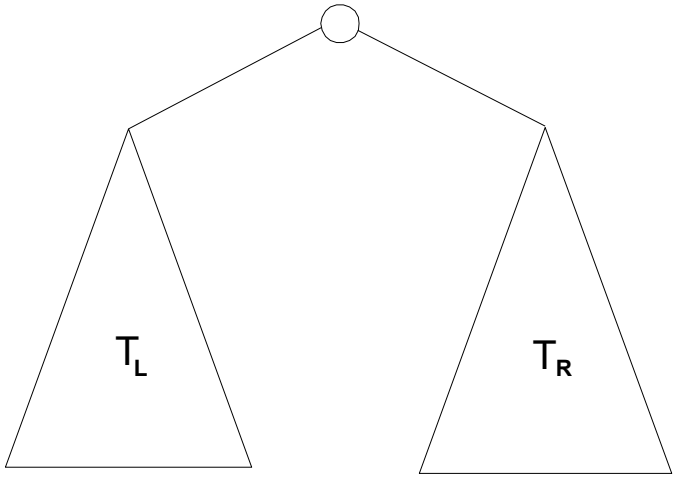


preorder: *a, b, d, g, e, c, f*
inorder: *d, g, b, e, a, f, c*
postorder: *g, d, e, b, f, c, a*

- Efficiency: $\Theta(n)$

Binary Tree Algorithms (cont.)

Ex. 2: Computing the height of a binary tree



ALGORITHM *Height*(T)

//Computes recursively the height of a binary tree
//Input: A binary tree T
//Output: The height of T
if $T = \varnothing$ **return** -1
else return $\max\{Height(T_{left}), Height(T_{right})\} + 1$

Efficiency: $\Theta(n)$

General Divide-and-Conquer Recurrence

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- an instance of size n can be divided into $\boldsymbol{b}$ instances of size $\boldsymbol{n/b}$, with $\boldsymbol{a}$ of them needing to be solved
- $\boldsymbol{f(n)}$ is a function that accounts for the time spent on dividing an instance and combining their solutions

<u>Master Theorem</u>: If $f(n) \in \Theta(n^d), \quad d \geq 0$

$$T(n) \in \begin{cases} \Theta\left(n^d\right) & if\ a < b^d \\ \Theta\left(n^d \log n\right) & if\ a = b^d \\ \Theta\left(n^{\log_b a}\right) & if\ a > b^d \end{cases}$$

Note: Analogous results hold for the O and Ω notations, too.

General Divide-and-Conquer Recurrence

Examples: the algorithm of divide-and-conquer sum-computation $(a_0 + \cdots + a_{n-1})$ on inputs of size $n = 2^k$

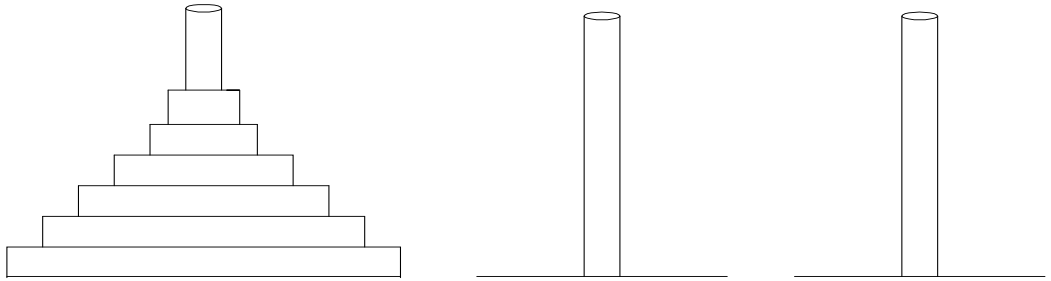
$$A(n) = 2A\left(\frac{n}{2}\right) + 1$$

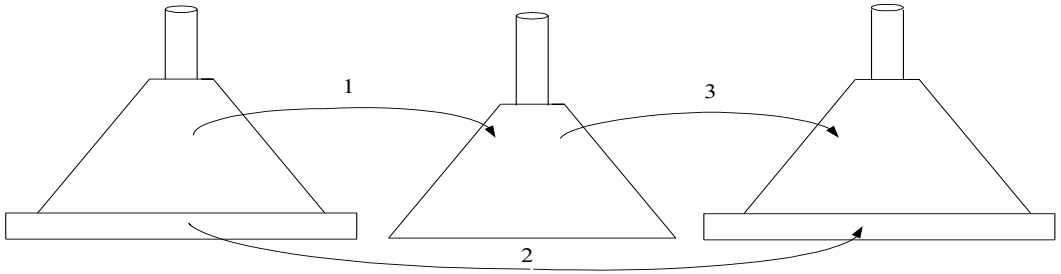
$a = 2, b = 2, d = 0$ hence $a > b^d$

$$A(n) \in \Theta\left(n^{\log_b a}\right) = \Theta(n^{\log_2 2}) = \Theta(n)$$

Example : The Tower of Hanoi Puzzle

To move n > 1 disks from peg 1 to peg 3 (with peg 2 as auxiliary),





First move recursively n − 1 disks from peg 1 to peg 2 (with peg 3 as auxiliary),

then move the largest disk directly from peg 1 to peg 3,

and, finally, move recursively n − 1 disks from peg 2 to peg 3 (using peg 1 as auxiliary).

Example: The Tower of Hanoi Puzzle

```
hanoi(n, a, b, c){
    if(n==0) return
    hanoi(n-1, a, c, b)
    print("move", n, a, "->", c)
    hanoi(n-1, b, a, c)
}
```

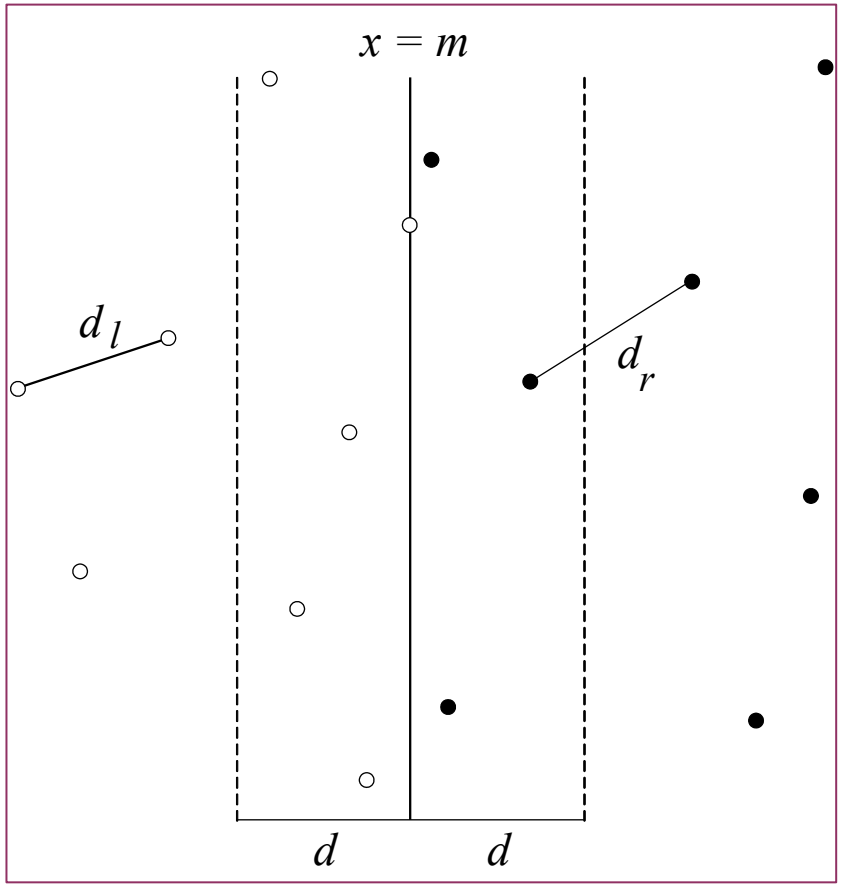
Closest-Pair Problem by Divide-and-Conquer

Step 1: Divide the points given into two subsets P_l and P_r by a vertical line x = m so that half the points lie to the left or on the line and half the points lie to the right or on the line.

Step 2 Find recursively the closest pairs for the left and right subsets.

Step 3 Set d = min{d_l, d_r}

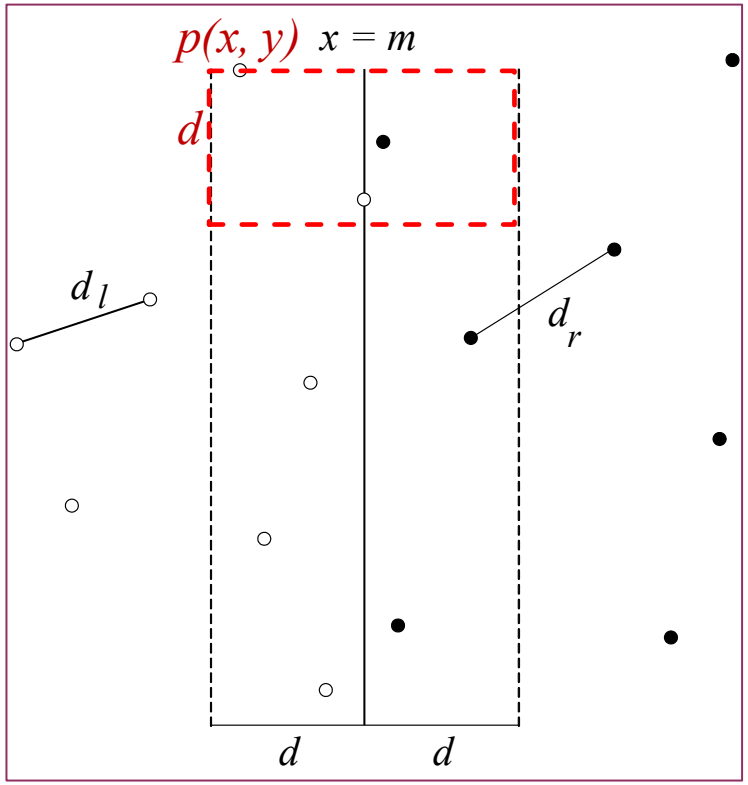
We can limit our attention to the points in the symmetric vertical strip S of width 2d as possible closest pair. (The points are stored and processed in increasing order of their y coordinates.)



Closest Pair by Divide-and-Conquer (cont.)

Step 4 Scan the points in the vertical strip S from the lowest up.

For every point p(x, y) in the strip, inspect points in the strip that may be closer to p than d. There can be no more than 5 such points following p on the strip list!



Efficiency of the Closest-Pair Algorithm

- Running time of the algorithm is described by

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

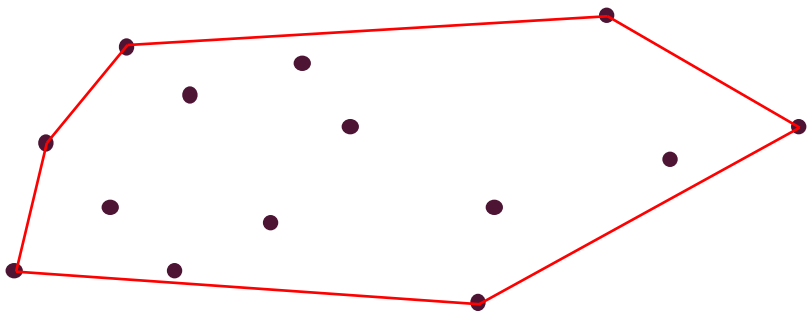
Case 2 of Master Method: $n \log n$

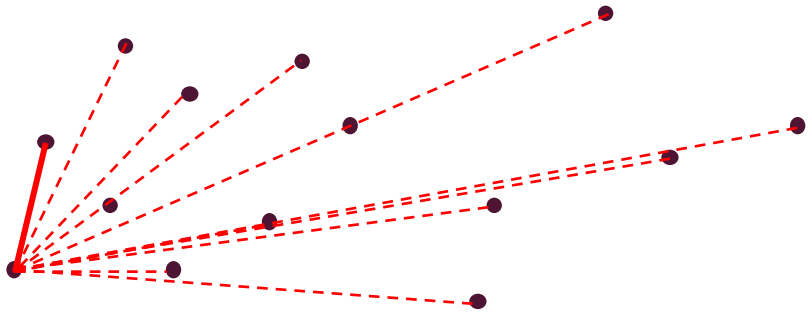
$$T(n) \in \Theta(n \log n)$$

- The necessity to presort input points does not change the overall efficiency class if sorting is done by a $O(n \log n)$ algorithm such as mergesort.

Convex hull Problem

- Given a set of points in the plane, the convex hull of the set is the smallest convex polygon that contains all the points of it.



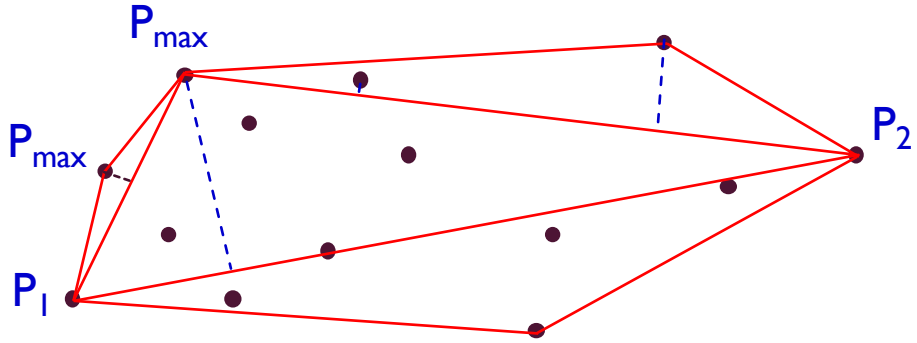


```
for all p1 in Points do
  for all p2 in Points do
    for all p3 in Points do
      if p3 is not lie on the same side with the other point of line p1p2
        p1p2 is not result
```

$\in \Theta(n^3)$

Quickhull Algorithm

- Assume points are sorted by *x*-coordinate values
- Identify *extreme points* P_1 and P_2 (leftmost and rightmost)
- Compute *upper hull* recursively:
  - find point P_{max} that is farthest away from line P_1P_2
  - compute the upper hull of the points to the left of line P_1P_{max}
  - compute the upper hull of the points to the left of line $P_{max}P_2$
- Compute *lower hull* in a similar manner



Efficiency of Quickhull Algorithm

- Finding point farthest away from line P_1P_2 can be done in linear time
- Time efficiency:
  - worst case: $\Theta(n^2)$ (as quicksort)
  - average case: $\Theta(n)$ (under reasonable assumptions about distribution of points given)

- If points are not initially sorted by x-coordinate value, this can be accomplished in $O(n \log n)$ time

- Several $O(n \log n)$ algorithms for convex hull are known

Exercise

1. Longest Sub-String
2. Max Sum Sub-Sequence

Exercise

- 679 - Dropping Balls
  https://onlinejudge.org/index.php?option=onlinejudge&page=show_problem&problem=620

- 714 - Copying Books
  https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=655

- 10077 - The Stern-Brocot Number System
  https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&category=33&page=show_problem&problem=1018